Airbus Amber

SKYWISE · AI ENABLEMENT

Skywise AI FDE: best practices

How to get real value from the AI engineer inside Skywise, safely and without wasting tokens

[Presenter] · [Team] · [Date]

From forward deployed engineers to an AI one

A forward deployed engineer

Sits with the business, understands the problem, and builds the pipelines, ontology and apps in Skywise to solve it.

AI FDE

An agent inside Skywise that does the same kind of work through conversation: you describe the outcome, it plans, uses Skywise's own tools, checks its work and proposes the change.

You

Stay in charge: you give context, approve actions, review the branch proposal or pull request, and decide what goes to production.

It works as you.
It uses your login and your permissions: if you can't do something in Skywise, neither can it. Everything is logged under your name.

It proposes, you dispose.
By default it works on a branch and hands you a Global Branch proposal or a Code Repository pull request to review.

A closed loop: plan, act, check, propose

1 Understand

Reads your request and the context you attached (datasets, object types, repos, docs)

→

2 Plan

Picks a mode, can ask clarifying questions and draft a plan for you to approve

→

3 Act

Uses Skywise tools: write a transform, create an object type, run a build

→

4 Check

Observes the result: transform and function previews, CI checks, errors; then fixes

→

5 Propose

Branch proposal or pull request, with an explanation of what changed

WHERE YOU COME IN

Before: give it the right context and, for anything non-trivial, ask for a plan first.

During: approve or reject tool actions. Read what it's about to do before you click approve.

After: review the proposal or pull request, test with real sample data, then merge.

Nine modes cover most of Skywise

Data integration
Build or change pipelines: Python transforms or Pipeline Builder

Data connection
Create, manage and debug sources and egress policies

Ontology editing
Create or update object types, links and actions

Functions editing
Write functions in Logic, TypeScript or Python; test them with AIP Evals

Exploration
Read-only investigation of what exists, including Workflow Lineage

Governance
Audit permissions, access, markings and data protection

Machine learning
Train, evaluate, deploy and tune models (no-code or pro-code)

OSDK React
Build React apps or custom widgets on Skywise data

Platform Q&A
Answer how-to questions about Skywise

What it can do, and the controls you have

AGENT CAPABILITIES

Generate plan
drafts a plan for you to review before acting

Request clarification
asks you questions (multiple choice or free text)

Load documentation
looks up Skywise docs when it needs them

Manage context
tidies its working memory in long sessions

Domain tools
filesystem, Notepad, solution design diagrams, execute actions on objects

YOUR CONTROLS

Context ribbon
attach datasets, functions, branches, object and action types, docs, media

Tools menu
switch on only the tools the task needs

Approvals
writes need your OK; reads run freely; edits on feature branches can be pre-approved

Chat outline
every prompt, response and tool used, with a token count per message

Compaction
summarise or remove old messages to stay within the context window

Pre-fill links and skills
a link (or a Workshop button) that opens a session with a prompt and AIP skills loaded

Eight habits that make it work

1. **One task, one session**
   Start a new session per task; long mixed sessions drift and cost more
2. **Pick the mode**
   Choose it yourself when you know it; it loads the right tools and docs
3. **Attach exactly what's needed**
   The specific dataset, object type or repo, not whole folders
4. **Ask for a plan first**
   For anything multi-step: review the plan, then let it act
5. **Small steps**
   Build the basic structure, verify, then add complexity
6. **Be specific about done**
   Name inputs, outputs, columns, rules and how you'll check it
7. **Verify on the branch**
   Read the code, run it on representative sample data, check the preview
8. **Reuse what works**
   Turn good prompts into AIP skills and pre-fill links for the team

Vague asks waste turns; specific asks get results

INSTEAD OF	WRITE
"Fix my pipeline"	"Build of [dataset] fails on the join step since yesterday. Find the cause from the build log, propose a fix on a branch, and show me the transform preview."
"Make an ontology for risks"	"Create object type Risk from [dataset]: primary key risk_id, title from risk_title, link to Programme via programme_id. Plan first; don't create actions yet."
"Clean this data"	"In a Python transform on [dataset], drop rows with null asset_id, trim whitespace in all string columns, output [new dataset]. Test on a 1,000-row sample."
"Who can see this?"	"Governance mode: list who has access to [dataset] and through which groups and markings. Read-only."

A good prompt names: the resource · the goal · the rules · what not to touch · how to verify

What it can't do, and shouldn't

BY DESIGN, IT CANNOT

Go beyond your permissions
no service account, no escalation; it sees and changes only what you can

Write without your consent
actions, publishing, apps and tags always need approval

Share a session
sessions are private to you, and follow your markings

Use models we haven't enabled
on our enrollment that means Gemini models only

Know what you didn't tell it
it starts with no user data; it only knows the context you add

AND YOU SHOULDN'T

Ship its output unreviewed
it's built for fast prototypes; production needs your review, tests and tuning

Approve without reading
check which tool and which action before every approval

Treat it as a domain expert
it knows Skywise, not our business rules: give it those

Run it at scale unchecked
it can fire dozens of builds in minutes; mind compute, GPUs and storage

Expect identical features everywhere
availability can differ by enrollment and changes over time

Every message re-reads the whole session

So the cost of a session grows with its length and with everything attached. Keep it small.

Start small, add on demand
Attach the one dataset or object type, not the project; let it search only if needed

Switch off unused tools
Fewer tools = shorter instructions on every turn and fewer wrong turns

New session per task
Don't carry yesterday's debugging into today's ontology change

Compact long sessions
Summarise or remove old messages once a step is done

Watch the outline
The per-message token count shows what's expensive: big attachments, long logs

Plan first
One reviewed plan is cheaper than three rounds of rework

Point, don't paste
Reference datasets and logs by resource; don't paste large tables or files

Ask for diffs and summaries
"Show only the changed lines" instead of whole files; "summarise the log"

Right Gemini for the job
A fast Flash model for Q&A and exploration; a stronger model for building

Reuse skills and links
Long standing instructions live in an AIP skill, not retyped every session

Pick the Gemini model by the job

We only use Gemini models. Choose from the ones enabled on our Skywise enrollment in the model picker.

Fast model (Flash)

Cheapest and quickest

- Platform Q&A
- Exploration and lineage questions
- Summarising logs and resources
- Governance look-ups

Strongest reasoning model (Pro / Thinking)

Slower, more tokens per turn, far fewer mistakes

- Pipelines and transforms
- Ontology design and actions
- Functions and Logic
- Debugging failed builds, multi-step plans

Rule of thumb Explore and ask with the fast model; switch to the strong model when it starts changing things. A cheap model that needs three tries costs more than a strong one that gets it right once.

Before, during and after every session

BEFORE

- ☐ New session, right mode
- ☐ Attach only the resources needed
- ☐ Switch off unused tools
- ☐ Gemini model to match the task
- ☐ Write the goal, rules and checks

DURING

- ☐ Ask for a plan; review it
- ☐ Read every approval before clicking
- ☐ Keep steps small
- ☐ Watch token counts; compact
- ☐ Stop it if it goes in circles

AFTER

- ☐ Review the proposal or pull request
- ☐ Test on representative sample data
- ☐ AIP Evals for functions
- ☐ Merge only what you understand
- ☐ Save good prompts as skills

An engineer you direct, not an autopilot

Small and specific

One task, the right mode, exactly the context it needs

Plan, then act

Review the plan and every approval

Verify before merge

Branch, sample data, AIP Evals

Mind the tokens

Short sessions, compaction, the right Gemini model

The full best-practice guide is shared with this deck: Skywise-AI-FDE-Best-Practices.docx

Questions?